

Ahmet Kazankaya

AI PLATFORM ENGINEER · AGENT INFRASTRUCTURE, EVALUATION AND GOVERNANCE

Istanbul, Turkey · open to relocation in Europe, remote works too · ahmetkazankaya1@gmail.com · +90 536 214 8114 · linkedin.com/in/ahmetkazankaya · github.com/kasparovabi

SUMMARY

I build the platform layer that makes LLM agent workflows run reliably in production: execution runtimes, human approval gates, permission boundaries, run logging and the evaluation harnesses that decide whether an agent is allowed to ship. I own the architecture end to end, from the first design decision to the pager. I came to software through production work rather than a computer science degree, and my process assumes model output is wrong until something independent proves otherwise.

Since February 2026 I have consulted for a foundation running schools in 66 countries, where I own the internal automation platforms used daily by non-engineering teams: 14 applications in six months, 12 of them live, running reliably at department scale without me in the loop. I also run the enablement, so the systems keep working without me in the loop. The LLM agents I develop run in production against real users and real money, not as prototypes. I create and maintain open source developer tools that other engineers install and run unchanged, and I ensure the same verification gates apply to my own work.

PLATFORM AND RELIABILITY WORK

Execution platform and durable workflows.

Event-triggered pipelines with scheduled and webhook entry points, durable state in Postgres and SQLite, job queues, retry and catch-up logic. The platform enables non-engineers to trigger reliable AI workflows without touching the runtime, and I monitor every run for failure and drift. A watchdog re-triggers runs that silently failed, capped at 3 attempts per day so a broken job cannot loop forever, because a scheduled job that dies quietly is worse than one that errors loudly. Every run is logged end to end and traceable back to its inputs, and failed runs trigger rollback rather than leaving half-written state behind.

Model-agnostic runtime and cost control.

Each task routes to the cheapest model that can complete it, across Anthropic, OpenAI and local Ollama, through OpenRouter and provider SDKs. Token spend, latency and failure rates are tracked per run, so cost and reliability are observable rather than estimated. A CI check validates every model identifier in the codebase against the live provider API, 10 models verified per run, so a retired model can never reach production. I have contributed this check upstream to open source agent tooling.

Evaluation layer.

audit-20 turns twenty recurring failure classes in AI-generated code into a CI gate of twenty behavioral checks that catches unreliable output during development rather than after release, with a final pass whose only job is to falsify the previous nineteen, plus a false-positive filter so real defects are not buried in noise. Safety-relevant classes are checked against golden outputs and monitored per run, so a safety regression fails the build instead of reaching a user. When the gate made output worse on a newer model generation, an evaluation harness running the same task 20 times with and without it proved the effect was real and model-dependent, so the scaffold is now gated by model rather than always on. That harness is the reason I trust the gate at all.

Governance and least privilege.

Governance mechanisms are built into the platform rather than written in a policy document. Risk tiers decide what needs review: high-risk actions require human approval, so nothing reaches a live storefront, a customer inbox or a public repository without a person approving it first. Access controls and secret management are enforced at the runtime boundary. Permissions are scoped per job with audit logging on every privileged call, tool access is granted per task rather than per agent, credentials are held outside the agent context, and infrastructure runs inside a Tailscale private network with no ports exposed to the internet. Restrictions are enforced by system design, not by prompts.

Security research and coordinated disclosure.

Two coordinated disclosures against open-source AI tooling, both filed privately through the vendor's own channel with no working exploit published. The first covered 17 vulnerabilities in a wearable AI device backend, aggregate CVSS 10.0, including remote code execution, authentication bypass, SSRF and a hardcoded production key; the vendor's own hardening commit names that disclosure in its title, so the outcome is verifiable in their git history rather than only in my write-up. The second reported that a documented filesystem allowlist did not exist in the shipped Rust binary, so the hardening step the project told operators to take did nothing.

I reproduce every finding locally before reporting, and I check each one against already-published advisories first: one prepared report was dropped unsent because an existing CVE already covered it. Reporting less is part of the discipline, since a duplicate costs a maintainer the same time as a real bug. Full timeline and artifacts: github.com/kasparovabi/security-research

User adoption and measurement.

Platforms are instrumented for usage, failure rates and cost, so impact across the organization is measurable rather than asserted. I ran one-to-one enablement for a team of 15 and am preparing a foundation-wide seminar, because adoption is the half of the work that decides whether any of it survives.

SELECTED SYSTEMS

Corporate task platform · foundation with schools in 66 countries · Mar 2026 to present · in production

The department head used to read every incoming request, decide who should handle it, and track delivery himself, for a team of 15. The platform ingests the mailbox, turns each message into a structured task with an LLM, assigns it, and displays the queue on an office board. Next.js, React, Supabase. Twelve of the department's fourteen internal applications are live and in daily use.

Obstacle: generated documents drifted from institutional templates and a text diff showed nothing, because the drift was visual. Comparing delivered files against fixed templates visually made template stability measurable.

guard-20 and audit-20 · open source · Jul 2026 to present · installed by other developers

Verification tooling for AI-generated code: rules that apply while the model writes, and a twenty-pass gate before delivery. github.com/kasparovabi

Demand and listing engine · own production business · Jun 2026 to present · running daily

Public-domain works and generated art become products and marketplace listings through a Telegram approval gate, on a schedule. A pricing engine pulls the fulfilment shipping table per variant and country and disables combinations that cannot be profitable.

Obstacle: agents kept producing near-duplicate ideas. The model was the obvious suspect and swapping it changed nothing; the cause was the shape of the data handed to them. The clearest case I have of an AI failure that is not an AI problem.

Multilingual content platform · Jun 2026 to present · running daily

Assistant infrastructure with persistent memory and 13 scheduled jobs, chaining research, generation, media production and publishing. Generated text is checked automatically against a written style specification before it can ship. Runs on AWS inside a Tailscale private network.

Distributed render pipeline · Feb 2026 to present · in production

Parametric video rendering where theme, camera movement and event data are parameters and jobs are processed from a queue. Orchestration on macOS, GPU-bound work on Windows, connected over a private network. Over 3,600 files, actively developed.

STACK

Languages: Python, TypeScript, JavaScript, SQL, Shell, Swift, C++

Agent and LLM: Claude Agent SDK, MCP servers, OpenRouter, Ollama, RAG pipelines, prompt and context distribution tooling

Backend and data: FastAPI, SQLAlchemy, PostgreSQL, SQLite, Supabase, Prisma, REST API integration

Cloud and infrastructure: AWS, Cloudflare Workers, Docker, Tailscale, launchd and cron, GitHub Actions, headless CI

Frontend: Next.js, React, Tailwind, SwiftUI

Testing and evaluation: pytest, vitest, Playwright, XCTest, custom evaluation harnesses, golden-output comparison

Integrations: Microsoft Graph and Outlook 365, Telegram, WhatsApp, marketplace and fulfilment APIs, Deepgram, ElevenLabs

EXPERIENCE

AI Solutions Designer · Türkiye Maarif Vakfı · consulting engagement · Feb 2026 to present

Generative AI Engineer · Pepper Root, Creative AI Studio · remote · Nov 2025 to Jan 2026

AI Artist · Joy Game · Istanbul · Aug 2025 to Oct 2025

Chief GenAI Art Director · GPT Journey · Istanbul · Mar 2024 to Aug 2025

Generative AI Creator · Fiverr · remote · Oct 2022 to Jun 2025

Education: Mechanical Engineering, Namık Kemal University, from 2012, left before finishing. Ten years in visual production before software.

Languages: Turkish (native), English (professional)